

PAYMENT TRANSACTION MANAGEMENT SYSTEM

Introduction:

The **Payment Transaction Management System** is designed to manage and maintain payment information related to customer transactions. The system stores payment details such as payment ID, order ID, payment date, payment amount, payment mode and payment status.

The system helps to maintain payment records accurately and provides useful reports for analysing payment transactions.

Creating Payment Table:

The **PAYMENT** table is used to store the details of each payment transaction. It contains information such as Payment ID, Order ID, Payment Date, Payment Amount, Payment Mode and Payment Status.

```
SQL> CREATE TABLE PAYMENT(Payment_ID VARCHAR(7),ORDER_ID VARCHAR(7),Payment_Date DATE,Payment_Mode VARCHAR(20),Payment_Status VARCHAR(10),Transaction_Amount NUMBER(10,2));  
Table created.
```

Applying Primary Key:

The **Payment_ID** is used as the primary key of the PAYMENT table. It uniquely identifies each payment transaction.

```
SQL> ALTER TABLE Payment ADD CONSTRAINTS pk_payment PRIMARY KEY (payment_ID);  
Table altered.
```

Applying Foreign Key:

The **Order_ID** is used to establish a relationship between the PAYMENT table and the ORDERS table.

This ensures that every payment transaction is associated with a valid order.

```
SQL> ALTER TABLE Payment ADD CONSTRAINTS fk_payment_order FOREIGN KEY (order_ID) REFERENCES Orders(order_ID);  
Table altered.
```

Applying NOT NULL Constraint:

The **NOT NULL** constraint is applied to important payment attributes to ensure that required payment information is not left empty.

For example, Payment ID, Order ID, Payment Amount and Payment Mode can be maintained as mandatory fields.

```
SQL> ALTER TABLE payment MODIFY(Payment_Status NOT NULL, Transcation_Amount NOT NULL);  
Table altered.
```

Applying CHECK Constraint:

The **CHECK** constraint is used to maintain valid payment values.

```
SQL> ALTER TABLE Payment ADD CONSTRAINTS chk_transcation_amount CHECK (Transcation_Amount > 0);  
Table altered.
```

```
SQL> ALTER TABLE Payment ADD CONSTRAINTS chk_payment_mode CHECK (payment_Mode IN('UPI','CARD','NET BANKING','CASH'));  
Table altered.
```

```
SQL> ALTER TABLE Payment ADD CONSTRAINTS chk_payment_Status CHECK (payment_Status IN('success','failed','pending'));  
Table altered.
```

Inserting Payment Records:

Payment transaction records are inserted into the PAYMENT table using SQL INSERT statements. Each payment record contains the required transaction details.

```
SQL> INSERT INTO PAYMENT VALUES ('PAY0015','0301',TO_DATE('07-10-2026','DD-MM-YYYY'),'UPI','success',1800.00);  
1 row created.  
SQL> INSERT INTO PAYMENT VALUES ('PAY0016','0302',TO_DATE('08-10-2026','DD-MM-YYYY'),'CARD','success',2500.00);  
1 row created.  
SQL> INSERT INTO PAYMENT VALUES ('PAY0017','0303',TO_DATE('09-10-2026','DD-MM-YYYY'),'NET BANKING','pending',3200.00);  
1 row created.
```

```

SQL> INSERT INTO PAYMENT VALUES ('PAY0019','0305',TO_DATE('11-10-2026','DD-MM-YYYY'),'UPI','pending',5200.00);
1 row created.

SQL> INSERT INTO PAYMENT VALUES ('PAY0020','0306',TO_DATE('12-10-2026','DD-MM-YYYY'),'CARD','failed',3600.00);
1 row created.

SQL> INSERT INTO PAYMENT VALUES ('PAY0021','0307',TO_DATE('13-10-2026','DD-MM-YYYY'),'NET BANKING','success',6100.00);
1 row created.

SQL> INSERT INTO PAYMENT VALUES ('PAY0022','0308',TO_DATE('14-10-2026','DD-MM-YYYY'),'CASH','success',1600.00);
1 row created.

SQL> INSERT INTO PAYMENT VALUES ('PAY0023','0309',TO_DATE('15-10-2026','DD-MM-YYYY'),'UPI','success',2800.00);
1 row created.

SQL> INSERT INTO PAYMENT VALUES ('PAY0024','0310',TO_DATE('16-10-2026','DD-MM-YYYY'),'CARD','pending',7500.00);
1 row created.

```

MANAGE SUCCESSFUL AND FAILED TRANSACTION

TASK:

- Display all successful payment
- Find failed transaction
- Count total successful and failed payment

```

SQL> SELECT * FROM payment WHERE PAYMENT_STATUS = 'success';

```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSCATION_AMOUNT
PAY0015	0301	07-OCT-26	UPI	success	1800
PAY0016	0302	08-OCT-26	CARD	success	2500
PAY0021	0307	13-OCT-26	NET BANKING	success	6100
PAY0022	0308	14-OCT-26	CASH	success	1600
PAY0023	0309	15-OCT-26	UPI	success	2800

```

SQL> SELECT * FROM payment WHERE PAYMENT_STATUS = 'failed';

```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSCATION_AMOUNT
PAY0020	0306	12-OCT-26	CARD	failed	3600

```

SQL> SELECT Payment_status,COUNT(Payment_status) FROM Payment GROUP BY Payment_status;

```

PAYMENT_ST	COUNT(PAYMENT_STATUS)
failed	1
success	5
pending	3

ANALYZE PAYMENT METHOD USED BY CUSTOMERS

REPORT 1: PAYMENT MODE ANALYSIS

Find:

- Number Of Transaction Using UPI.
- Number Of Card Payment.
- Most Preferred Payment Method.

```
SQL> SELECT COUNT(*) AS UPI_TRANSCATION FROM PAYMENT WHERE PAYMENT_MODE = 'UPI';

UPI_TRANSCATION
-----
                3

SQL> SELECT COUNT(*) AS CARD_PAYMENT FROM PAYMENT WHERE PAYMENT_MODE IN ('Credit Card','Debit Card');

CARD_PAYMENT
-----
            0

SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE      TRANSACTION_COUNT
-----
UPI                3
CARD               3
NET BANKING        2
CASH               1

SQL> SELECT * FROM (SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC) WHERE ROW
NUM <= 2;

PAYMENT_MODE      TRANSACTION_COUNT
-----
CARD               3
UPI                3
```

REPORT 2: REVENUE ANALYSIS

Find:

- Total Revenue Generated.
- Revenue By Payment Method.
- Average Transaction Amount.

```
SQL> SELECT * FROM (SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC) WHERE ROW
NUM <= 2;

PAYMENT_MODE      TRANSACTION_COUNT
-----
CARD               3
UPI                3

SQL> SELECT SUM(TRANSCATION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'success';

TOTAL_REVENUE
-----
        14800
```

```
SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TOTAL_REVENUE
NET BANKING	6100
UPI	4600
CARD	2500
CASH	1600

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT),2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS = 'success';
```

AVERAGE_TRANSACTION_AMOUNT
2960

REPORT 3: CUSTOMER PAYMENT HISTORY

Display:

- Customer Name
- Order ID
- Payment Mode
- Amount
- Payment Status

```
SQL> SELECT C.CUSTOMER_NAME AS CUSTOMER_NAME, O.ORDER_ID AS ORDER_ID, O.ORDER_DATE AS ORDER_DATE, P.PAYMENT_DATE AS PAYMENT_DATE, P.PAYMENT_MODE AS PAYMENT_MODE, P.TRANSACTION_AMOUNT AS AMOUNT, P.PAYMENT_STATUS AS PAYMENT_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID JOIN PAYMENT P ON O.ORDER_ID=P.ORDER_ID ORDER BY P.PAYMENT_DATE;
```

CUSTOMER_N	ORDER_ID	ORDER_DATE	PAYMENT_DA	PAYMENT_MODE	AMOUNT	PAYMENT_ST
AJAY	0301	20-SEP-26	07-OCT-26	UPI	1800	success
kavya	0302	21-SEP-26	08-OCT-26	CARD	2500	success
MANOJ	0303	22-SEP-26	09-OCT-26	NET BANKING	3200	pending
RAVI	0305	24-SEP-26	11-OCT-26	UPI	5200	pending
AJAY	0306	24-SEP-26	12-OCT-26	CARD	3600	failed
kavya	0307	25-SEP-26	13-OCT-26	NET BANKING	6100	success
MANOJ	0308	25-SEP-26	14-OCT-26	CASH	1600	success
NISHA	0309	26-SEP-26	15-OCT-26	UPI	2800	success
RAVI	0310	26-SEP-26	16-OCT-26	CARD	7500	pending

9 rows selected.

PAYMENT ANALYSIS:

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE, ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS = 'success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
NET BANKING	1	6100	6100
UPI	2	4600	2300
CARD	1	2500	2500
CASH	1	1600	1600